

Inteligência Artificial
Prof. Bruno M. Nogueira
Faculdade de Computação - UFMS
Pós-Graduação - 2026/1

Trabalho Prático

Construindo o “R” do RAG: um sistema de recuperação de artigos científicos

1. Contextualização e motivação

A explosão na produção científica fez com que o gargalo da pesquisa deixasse de ser a escrita e passasse a ser a *descoberta* de trabalhos relevantes. Plataformas como o ArXiv recebem milhares de submissões por mês, e ferramentas modernas baseadas em **Retrieval-Augmented Generation (RAG)** têm sido propostas para auxiliar pesquisadores a navegar nesse volume. Em um sistema RAG, antes que qualquer modelo generativo entre em cena, é preciso *recuperar* um pequeno conjunto de documentos potencialmente relevantes para a consulta do usuário - e é justamente esse componente, o “R” do RAG, que será o objeto deste trabalho.

A boa notícia é que o “R” do RAG é, em essência, um problema de **aprendizado de máquina** sobre texto: tudo o que vimos no semestre - ponderação probabilística (Naïve Bayes), classificação (Regressão Logística, SVM, Árvores de Decisão), busca por vizinhos mais próximos (KNN), agrupamento de dados, regras de associação e algoritmos de otimização - pode ser combinado em um sistema funcional que ajuda um pesquisador a encontrar artigos interessantes em uma base científica real.

2. Objetivo

O objetivo deste trabalho é **projetar, implementar e avaliar** um sistema de recuperação de artigos científicos sobre uma coleção construída por você a partir do ArXiv (ou outra base científica equivalente; ver Seção 5). O sistema deve receber como entrada uma *consulta em texto livre* (e.g., “*self-supervised graph representation learning for molecular property prediction*”) e devolver uma lista ranqueada de artigos da coleção, juntamente com uma análise crítica do desempenho de diferentes estratégias de recuperação estudadas em sala.

A Figura 1 resume, em alto nível, o *pipeline* esperado: a partir das definições do seu tema de pesquisa, uma coleção é construída e pré-processada; sobre ela são implementados *dois* recuperadores diferentes (BM25 e KNN/denso), obrigatoriamente; opcionalmente, módulos de aprofundamento podem ser acoplados; o sistema é então avaliado com consultas e *relevance judgments* criados por você; e a entrega final é composta pelo relatório, código e vídeo.

3. Pipeline obrigatório (todos os alunos)

O sistema deve contemplar, no mínimo, as cinco etapas a seguir. **Todas as cinco são obrigatórias - inclusive ter *dois* recuperadores diferentes (BM25 e KNN/denso) rodando sobre a mesma coleção.** Os módulos da Seção 4 só entram em cima desse pipeline.

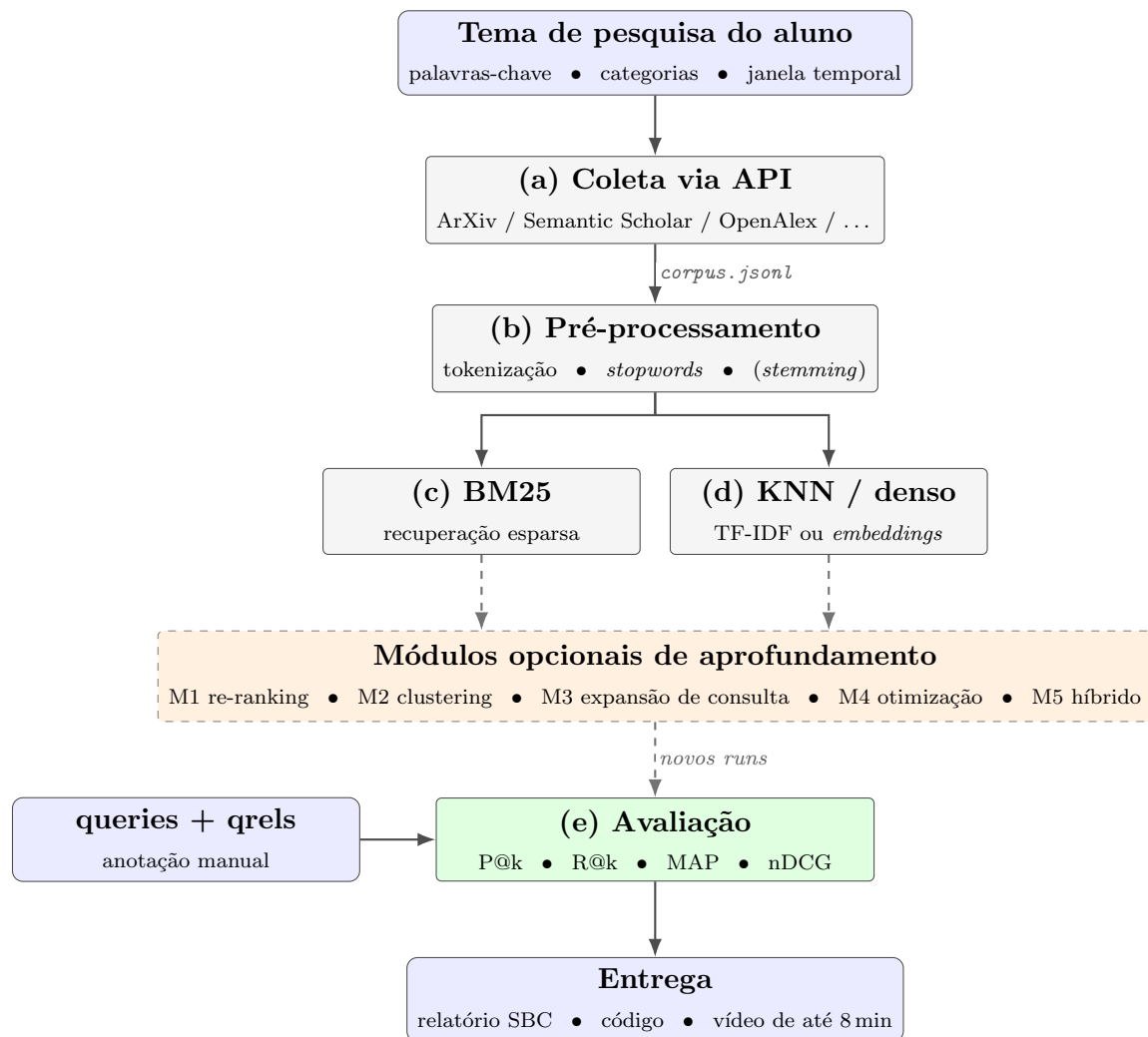


Figura 1: Visão geral do *pipeline* do trabalho. **Entrada** (azul, topo): tema de pesquisa do aluno traduzido em palavras-chave, categorias e janela temporal. **Etapas obrigatórias** (cinza, setas sólidas): coleta da coleção, pré-processamento e *dois* recuperadores (BM25 esparsa e KNN/denso) executados em paralelo - ambos alimentam diretamente a avaliação. **Módulos opcionais** (laranja, tracejado - setas tracejadas): re-ranqueamento, agrupamento, expansão de consulta, otimização e ranking híbrido. Quando usados, recebem os rankings de BM25/KNN, produzem *novos runs*, e estes também são avaliados. **Avaliação** (verde): cruza cada *run* com as consultas e *relevance judgments* (azul, à esquerda) construídos manualmente pelo aluno (ver Seção 3-e). **Saída** (azul, base): relatório SBC, código e vídeo.

Fluxo de trabalho recomendado. Não comece tudo de uma vez. Recomendamos a ordem abaixo, com indicação do material de apoio que destrava cada etapa (Seção 10):

- Passo 1:** Defina *no papel* o tema da coleção, as palavras-chave e a janela temporal → etapa (a). *Material:* exemplo na Seção 3-(a).
- Passo 2:** Rode o notebook `coleta_arxiv.ipynb` e gere o `corpus.jsonl` → etapas (a) e (b).
- Passo 3:** Rode o notebook `baseline_bm25.ipynb` e obtenha o primeiro *run* → etapa (c).
- Passo 4:** Implemente o recuperador KNN/denso (TF-IDF ou *embeddings*) → etapa (d).
- Passo 5:** Crie 10–20 *queries* de teste, faça *pooling* e anote o `qrels.tsv` manualmente → etapa (e).
- Passo 6:** Use `eval/evaluate.py` para gerar as métricas comparando os dois *runs* → etapa (e).
- Passo 7:** Só então, escolha e implemente o(s) módulo(s) da Seção 4.
- Passo 8:** Escreva o relatório *enquanto* avalia (não deixe para o final) e grave o vídeo por último.

(a) **Coleta e construção da coleção.** Construa uma coleção com **entre 1.000 e 5.000 artigos** extraídos da API pública do ArXiv (ou base equivalente). Para cada artigo, armazene ao menos: `id`, `título`, `abstract`, `autores`, `categorias` e `data`.

O tema da coleção é parte essencial do trabalho e deve refletir o seu projeto de pesquisa (mestrado ou doutorado). A motivação é dupla: (i) você terminará o semestre com um *retriever* útil para a sua própria revisão bibliográfica; e (ii) ao conhecer o domínio, você consegue julgar com propriedade a relevância dos resultados retornados na hora de construir o conjunto de avaliação (Seção 3-e).

Exemplo de como definir o escopo da coleção a partir do seu tema de pesquisa:

- **Tema de pesquisa do aluno:** “Detecção de fraudes em transações financeiras com modelos de aprendizado profundo”.
- **Tradução em termos/palavras-chave:** *fraud detection, anomaly detection, credit card fraud, financial transactions, graph neural networks for fraud.*
- **Categorias do ArXiv compatíveis:** `cs.LG`, `cs.CR`, `stat.ML`.
- **Janela temporal escolhida:** 2018–2026 (justificativa: explosão de trabalhos com *deep learning* nesse intervalo).
- **Estratégia de coleta:** consulta ao ArXiv combinando categorias e palavras-chave no título / abstract, paginando até obter ≈ 2.000 artigos e removendo duplicatas por DOI ou por `arxiv_id`.
- **Exemplos de queries de avaliação (Seção 3-e):** “self-supervised learning for credit card fraud”, “graph neural networks for money laundering detection”, “concept drift in fraud detection”.

O relatório deve documentar essa decisão explicitamente em uma subseção da metodologia (e.g., “*Escopo da coleção e relação com o projeto de pesquisa*”). Caso o seu projeto de pesquisa não esteja em uma área coberta pelas bases científicas indicadas (Seção 5), procure o professor para combinar uma alternativa.

(b) **Pré-processamento textual.** Aplique e justifique escolhas de tokenização, normalização (*lower-casing*, remoção de pontuação), remoção de *stopwords* e, opcionalmente, *stemming*/lematização. Discuta como cada escolha pode impactar a representação dos documentos.

(c) **Recuperação esparsa - BM25** (obrigatório). BM25 atribui a cada documento um *score* que é alto quando o documento contém os *termos da consulta* muitas vezes (TF), e mais alto ainda quando esses termos são *raros no corpus* (IDF), com correções pelo tamanho do documento. O documento com maior *score* fica no topo do ranking.

Implemente um **recuperador BM25** sobre o texto concatenado de título + abstract de cada artigo. É permitido o uso de bibliotecas (`rank_bm25`, `Pyserini`, etc.) - não é necessário programar a fórmula do zero. Você deve, contudo, **explicitar e justificar os dois hiper-parâmetros**:

- k_1 (**saturação da frequência do termo**): controla *quanto* uma alta frequência de um termo continua aumentando o *score* do documento. Valores típicos: 1.2 a 2.0. Quanto maior k_1 , mais peso para repetições; quanto menor, mais rápido a contribuição de um termo *satura*.
- b (**normalização por tamanho do documento**): $b \in [0, 1]$ controla *quanto* o tamanho do documento penaliza o *score*. $b = 0$: nenhuma penalização (documento longo vs. curto recebem a mesma escala); $b = 1$: penalização máxima. Padrão da literatura: $b = 0,75$.

No relatório, reporte os valores adotados, justifique-os (pode ser empiricamente, via experimento, ou citando o padrão da literatura) e **relacione a fórmula BM25 com o paradigma probabilístico discutido na aula de Naïve Bayes** - ambos modelam relevância como probabilidade, assumindo independência entre termos.

(d) **Recuperação por vizinhos mais próximos - KNN** (obrigatório). Implemente uma segunda estratégia, agora baseada na ideia da aula de **KNN**, mas aplicada à *recuperação* em vez de classificação. A diferença com o que vimos em sala é *conceitual* e não algorítmica:

KNN na aula (classificação)	KNN aqui (recuperação)
Exemplos rotulados $\rightarrow$ vetores.	Documentos da coleção $\rightarrow$ vetores.
Novo exemplo $\rightarrow$ vetor.	Consulta do usuário $\rightarrow$ vetor.
Encontram-se os K exemplos mais próximos e <i>vota-se a classe</i> .	Encontram-se os K documentos mais próximos e <i>devolvem-se como ranking</i> .
Saída: rótulo predito.	Saída: lista ranqueada dos K documentos mais similares.

Em outras palavras, em vez de *usar* os vizinhos para classificar, você *entrega* os vizinhos ao usuário como resposta à consulta. Concretamente:

1. Para cada documento d da coleção, calcule um vetor $\mathbf{v}_d \in \mathbb{R}^n$ (em geral, n varia de poucas centenas a alguns milhares, a depender da representação).
2. No momento da consulta, calcule o vetor $\mathbf{v}_q$ da *mesma forma*, usando o *mesmo* mapeamento texto $\rightarrow$ vetor.
3. Defina uma medida de *similaridade* $\text{sim}(\mathbf{v}_q, \mathbf{v}_d)$ entre vetores. A escolha mais comum em IR é a **similaridade do cosseno**:

$$\text{sim}_{\cos}(\mathbf{v}_q, \mathbf{v}_d) = \frac{\mathbf{v}_q \cdot \mathbf{v}_d}{\|\mathbf{v}_q\| \|\mathbf{v}_d\|}.$$

Distância euclidiana (vista em aula) também é válida; discuta a escolha no relatório.

4. Retorne os K **documentos com maior similaridade** à consulta. Esse é o seu *ranking* do recuperador denso.

Você pode escolher entre duas formas de obter os vetores:

- **Representação clássica via TF-IDF (recomendada como baseline).** TF-IDF é uma forma de transformar um texto em um vetor numérico onde *cada dimensão corresponde a um termo do vocabulário*, e o valor da dimensão é maior quando o termo aparece muito *naquele texto* (TF) e é raro no corpus inteiro (IDF). Textos sobre o mesmo assunto terminam com valores parecidos nas mesmas dimensões e, portanto, vetores próximos.

Como fazer: use o `TfidfVectorizer` do `scikit-learn` para vetorizar documentos e consultas, e o `NearestNeighbors` (ou `cosine_similarity`) para a busca. A conexão com a disciplina é direta: ponderação de termos (Naïve Bayes) + busca por similaridade (KNN).

- **Representação densa pré-treinada via *embeddings* (opcional).** *Embedding* é um vetor *denso* de dimensão fixa (tipicamente 384–768) produzido por uma rede neural pré-treinada, em que textos com *significado parecido* resultam em vetores próximos - mesmo quando não compartilham palavras exatas (e.g., “carro” e “automóvel”). Isso supera uma limitação clássica do TF-IDF, que depende de coincidência de termos.

Como fazer: use modelos prontos - por exemplo, `all-MiniLM-L6-v2` (da biblioteca `sentence-transformers`) ou `SPECTER2` (treinado especificamente sobre artigos científicos) - para gerar os vetores. O uso desses modelos é permitido (a aula não cobre o treinamento deles); a *busca* (cômputo das similaridades e seleção do top- K) deve ser implementada por você.

Observação importante - escolha de K . Em IR, K é o *tamanho do ranking devolvido* (e.g., $K = 100$ para gerar a *run* para avaliação, ou $K = 10$ se você for re-ranquear). *Não confunda* com o k do KNN-classificador da aula, que costuma ser pequeno (3, 5, 7). Aqui o K é tipicamente grande no estágio de recuperação e depois cortado para análise.

Síntese - o que fazer com os dois recuperadores. Após as etapas (c) e (d), você terá **dois sistemas independentes** que respondem à mesma consulta. Eles *não* são combinados nesta fase obrigatória - cada um produz seu próprio ranking, e a comparação entre eles é o **cerne experimental** do trabalho. Especificamente:

- Para *cada* consulta do conjunto de avaliação (etapa (e)), rode o BM25 e o KNN/denso *separadamente*, obtendo duas listas ranqueadas (*runs*).
- Salve cada *run* em um arquivo no formato TREC (e.g., `runs/bm25.trec` e `runs/knn.trec`). O notebook `baseline_bm25.ipynb` disponibilizado mostra como.
- Calcule as métricas (etapa (e)) *para cada run* separadamente.
- Discuta no relatório: *onde cada um vence?* Em que tipo de consulta o BM25 é melhor (e por quê)? E o KNN/denso? Há queries onde os dois falham? Essa análise **comparativa** é o que evidencia que você entendeu os *trade-offs* entre os paradigmas esparsos e densos vistos na disciplina.

Os **módulos da Seção 4** entram *depois* disso: eles *modificam* ou *combinam* os dois recuperadores (re-ranqueamento, ranking híbrido, agrupamento dos resultados, expansão da consulta, otimização de hiperparâmetros), produzindo *novos runs* que serão também avaliados e comparados com os dois baselines obrigatórios.

(e) Avaliação experimental. Construa manualmente um pequeno conjunto de avaliação: **entre 10 e 20 consultas** representativas, cada uma com *relevance judgments* (anotações de relevância) sobre os *top-k* resultados retornados por seus modelos. Detalhes a seguir.

Query de coleta vs. queries de teste — duas coisas diferentes. A query usada para *construir* a coleção e as queries usadas para *avaliar* os recuperadores têm propósitos opostos e **não devem ser confundidas**:

- A **query de coleta** (etapa (a)) é **genérica e ampla**. O objetivo dela é trazer *o máximo* de artigos potencialmente sobre o seu tema para dentro da coleção — tipicamente é um grande OR de palavras-chave abrangentes combinado com filtros de categoria e janela temporal. Você usa esta query **uma única vez**, na hora de baixar o corpus, e ela *não* entra na avaliação. Exemplo (tema “fraudes financeiras com DL”): (`‘‘fraud detection’’ OR ‘‘anomaly detection’’ OR ‘‘money laundering’’ OR ...`) AND (`cat:cs.LG OR cat:cs.CR`), no período 2018–2026.
- As **queries de teste** (10 a 20) são **específicas e focadas**. Cada uma simula uma *pergunta real* que um pesquisador da área faria a um buscador — uma frase curta, em linguagem natural, sobre um tópico bem definido. Elas são a *entrada* dos seus recuperadores na hora de avaliar, e *cada uma* terá seu próprio conjunto de documentos relevantes anotados (no `qrels`). Exemplos no mesmo tema:
 - “graph neural networks for credit card fraud detection”
 - “self-supervised learning for anomaly detection in financial transactions”
 - “concept drift in online fraud detection”
 - “surveys on deep learning for money laundering detection”
 - “explainability of fraud detection models”

Em uma analogia: a query de coleta *constrói a biblioteca* (genérica, traz tudo da área); as queries de teste são as *perguntas específicas que pesquisadores fariam a um bibliotecário* sobre o acervo dessa biblioteca.

O que é qrels? qrels é abreviação de *query relevance judgments* (“julgamentos de relevância por consulta”), termo padrão em Recuperação de Informação herdado dos *benchmarks* TREC. É, na prática, o **gabarito da sua avaliação**: para cada query de teste, você lista quais documentos da coleção são relevantes (e, opcionalmente, quão relevantes). Sem qrels não há como calcular nenhuma métrica de IR - o sistema devolve um ranking, mas não existe referência para saber se ele acertou ou errou. A próxima subseção descreve como construir o seu qrels de forma eficiente.

Boas práticas na escolha das queries de teste. (i) Cubra subáreas distintas do tema (e.g., métodos, aplicações, *surveys*, datasets); (ii) varie o nível de especificidade (algumas amplas, outras finas); (iii) evite usar literalmente as palavras-chave da query de coleta como *únicas* queries de teste; (iv) cada query deve ter, idealmente, *pelo menos um* documento relevante na sua coleção (caso contrário, não é avaliável).

Construção do qrels via *pooling*. “*Pooling*” é uma técnica clássica para reduzir o esforço de anotação: para cada query de teste, rode *todos* os seus modelos (BM25, KNN/denso, módulos), *uma* o *top-k* retornado por cada um (com k tipicamente = 10 ou 20) em um *pool* de candidatos, e **anote manualmente** cada documento desse *pool* como: 2 = altamente relevante, 1 = relevante, 0 = não relevante. Para relevância binária, use apenas 0/1. Documente o seu critério de relevância no relatório (e.g., “considero relevante todo artigo que responda diretamente à query”).

Formato do arquivo qrels.tsv (padrão TREC): cada linha do arquivo tem quatro componentes, separados por TAB: qid<TAB>iter<TAB>doc_id<TAB>rel, onde qid é o identificador da query (e.g., q1), doc_id é o identificador do documento na sua coleção (e.g., o *arxiv_id*) e rel é a relevância anotada (0/1 binária, ou 0/1/2 graduada). O campo do meio (iter) é um *vestígio histórico* do TREC: originalmente indicava em qual “rodada de anotação” aquele julgamento foi feito, prevendo a possibilidade de várias rodadas de anotação independentes para a mesma query. Na prática, quase ninguém usou múltiplas rodadas e **a convenção até hoje é deixar sempre 0 nesse campo** - os parsers (incluindo o nosso *evaluate.py*) simplesmente ignoram o valor; ele só está ali para manter compatibilidade com ferramentas antigas. Exemplo:

```
q1 0 2105.04054 2
q1 0 2206.03721 1
q1 0 2003.12345 0
q2 0 2210.11111 2
```

Formato do arquivo de *run* (TREC): qid Q0 doc_id rank score system. Uma linha por documento retornado, ordenadas por *rank* (1 = primeiro). Exemplo:

```
q1 Q0 2105.04054 1 18.5421 bm25
q1 Q0 2206.03721 2 16.7211 bm25
q1 Q0 2003.12345 3 12.0102 bm25
```

O script *eval/evaluate.py* fornecido lê esses dois formatos e calcula todas as métricas abaixo.

Métricas a reportar. Considere que, para uma dada query, o sistema devolve uma lista ranqueada $d_1, d_2, \dots, d_n$ de documentos da coleção. Para cada documento d_i , $\text{rel}(d_i)$ é a anotação de relevância do qrels (0/1 no caso binário, 0/1/2 no caso graduado). Dizemos que “ d_i é relevante” quando $\text{rel}(d_i) > 0$.

Exemplo de uso ao longo desta subseção: suponha uma query com $R = 4$ documentos relevantes no qrels (toda a coleção). O sistema devolveu, nas 5 primeiras posições, a sequência de relevâncias [1, 0, 1, 0, 1] (3 relevantes, 2 não-relevantes).

- **Precision@k (P@k).** Fração dos k primeiros documentos do ranking que são relevantes:

$$P@k = \frac{\text{\#relevantes entre as } k \text{ primeiras posições}}{k}.$$

Mede *qualidade do topo do ranking*. Útil quando o usuário olha apenas os primeiros resultados.

No exemplo: $P@5 = 3/5 = 0,60$.

- **Recall@k (R@k).** Fração dos *todos os documentos relevantes da coleção* (para aquela query) que aparecem nos k primeiros do ranking:

$$R@k = \frac{\text{\#relevantes entre as } k \text{ primeiras posições}}{R},$$

onde R é o total de documentos relevantes daquela query no **qrels**. Mede *cobertura*. Note que, com **qrels** via *pooling*, R é uma *estimativa* (você só conhece os relevantes que apareceram nos *pools*) - discuta essa limitação no relatório.

No exemplo: $R@5 = 3/4 = 0,75$.

- **Average Precision (AP) e Mean Average Precision (MAP).** A AP de uma query é a *média das precisões calculadas nas posições onde apareceu um documento relevante*. A MAP é a média das APs sobre todas as queries de teste. Em palavras: a AP premia rankings que colocam os relevantes mais ao topo.

$$AP = \frac{1}{R} \sum_{i: \text{rel}(d_i) > 0} P@i, \quad MAP = \frac{1}{|Q|} \sum_{q \in Q} AP_q.$$

MAP é a métrica *padrão* para comparar sistemas de IR com relevância binária.

No exemplo: relevantes ocorrem nas posições 1, 3 e 5. Logo: $AP = \frac{1}{4}(P@1 + P@3 + P@5) = \frac{1}{4}(1,00 + 0,67 + 0,60) \approx 0,57$.

- **nDCG@k.** Para relevância *graduada*, posições no topo pesam mais. Define-se o *Discounted Cumulative Gain* acumulando os *gains* (= valores de relevância) com desconto logarítmico pela posição:

$$DCG@k = \sum_{i=1}^k \frac{\text{rel}(d_i)}{\log_2(i+1)}.$$

O **nDCG@k** normaliza pelo melhor DCG possível (o *ideal*, calculado ordenando todos os documentos do **qrels** por relevância decrescente):

$$\text{nDCG@k} = \frac{DCG@k}{IDCG@k} \in [0, 1].$$

É a métrica recomendada quando você anotou relevância em *mais de dois níveis* (e.g., 0/1/2).

O que reportar: no *mínimo*, P@10, R@10, MAP e (se usou relevância graduada) nDCG@10. Reporte os valores *por modelo* e, no caso de Doutorado, com teste estatístico de significância (*paired t-test* ou Wilcoxon) na comparação principal. Apresente também **análise qualitativa de pelo menos duas queries** (acertos, falhas e por quê) - ver o material `eval/evaluate.py` disponibilizado, que já implementa todas essas métricas.

4. Módulos de aprofundamento (escolha conforme nível)

Além do pipeline obrigatório, escolha **módulos adicionais** dentre os listados a seguir. Cada módulo deve ser implementado, avaliado e discutido no relatório.

- **(M1) Re-ranqueamento por classificação supervisionada.** Treine um classificador de relevância (Regressão Logística, Naïve Bayes, Árvore de Decisão, SVM ou Rede Neural rasa) usando *features* extraídas dos pares (consulta, documento) - e.g., score BM25, similaridade do cosseno, sobreposição de termos, ano. Use o classificador para *re-ranquear* o *top-100* do BM25 e compare com o ranking original.
- **(M2) Agrupamento dos resultados.** Aplique algoritmos de agrupamento (K-means, Agrupamento Hierárquico, etc.) sobre o *top-k* retornado por uma consulta e proponha uma *visualização em facetas/tópicos*. Discuta a escolha do número de grupos e métricas de qualidade do agrupamento.
- **(M3) Expansão de consulta via regras de associação.** Trate cada documento como uma “transação” contendo o conjunto de seus termos (após pré-processamento) e minere regras do tipo “ $\{termoA\} \Rightarrow \{termoB\}$ ” com suporte/confiança acima de um limiar. Em tempo de consulta, expanda a query original com os termos consequentes das regras cujo antecedente bate com termos da query. *Exemplo:* se “transformer” implica frequentemente “attention”, uma query “transformer for fraud” poderia ser expandida para “transformer attention for fraud”. Avalie o impacto na recuperação (compare com BM25 sem expansão).
- **(M4) Otimização de hiperparâmetros.** Escolha um conjunto de hiperparâmetros do seu sistema (e.g., k_1 e b do BM25, peso α da combinação *sparse+dense* em um ranking híbrido, tamanho do *top-k* usado pelo módulo de re-ranking) e use um algoritmo de otimização visto na disciplina (*grid search*, *random search*, algoritmos genéticos, recozimento simulado, etc.) para ajustá-los. *Função objetivo sugerida:* MAP ou nDCG@10 sobre o seu conjunto de avaliação. Reporte os valores testados, a melhor configuração encontrada, e a curva ou tabela de desempenho.
- **(M5) Ranking híbrido *sparse+dense*.** Combine os *scores* de BM25 e do recuperador denso (e.g., por soma ponderada ou *Reciprocal Rank Fusion*) e compare contra cada componente isolado.

Quantidade mínima de módulos

- **Alunos de Mestrado:** pipeline obrigatório + **1 módulo** à sua escolha.
- **Alunos de Doutorado:** pipeline obrigatório + **2 módulos** à sua escolha, sendo obrigatoriamente um deles o M1 (re-ranqueamento) **ou** o M5 (ranking híbrido). Além disso, devem reportar **teste estatístico de significância** (e.g., *paired t-test* ou Wilcoxon) na comparação das melhores configurações.

5. Fontes de dados permitidas

A coleção pode ser construída a partir de uma das seguintes fontes (ou outra equivalente, aprovada pelo professor): API do **ArXiv** (<https://info.arxiv.org/help/api/index.html>), **Semantic Scholar API**, **OpenAlex**, **PubMed/PMC** ou **DBLP**. Não é permitido utilizar coleções pré-prontas de *benchmarks* de IR (como BEIR/MS MARCO) como a coleção principal - parte do trabalho é construir e documentar a sua própria coleção. Tais *benchmarks* podem, contudo, ser citados na revisão e usados para inspirar a sua metodologia.

6. Conexões com a disciplina

Você deve, ao longo do relatório, **explicitar as conexões** entre cada componente implementado e os conceitos vistos em sala. Sugestões de conexão (não exaustivas):

Componente do trabalho	Tópico da disciplina
BM25 / TF-IDF	Naïve Bayes / paradigmas estatísticos
KNN sobre vetores	Aula de Classificador KNN
Re-ranqueamento (M1)	Regressão Logística, SVM, Árvores, Naïve Bayes
Agrupamento de resultados (M2)	Aulas de Agrupamento de Dados (1, 2 e 3)
Expansão de consulta (M3)	Regras de Associação
Otimização de hiperparâmetros (M4)	Algoritmos de Otimização
<i>Embeddings</i> densos (opcional)	Redes Neurais e Backpropagation

7. Entregáveis

1. **Relatório em LaTeX (PDF).** Artigo no formato da SBC¹, com até **10 páginas** incluindo figuras e referências. O relatório deve conter, no mínimo: Resumo/Abstract, Introdução, Trabalhos Relacionados, Metodologia (coleta, pré-processamento, modelos implementados), Avaliação Experimental, Resultados e Discussão, Conclusão e Referências.
2. **Código-fonte e dados.** Repositório (Git/zip) contendo: scripts de coleta, código dos modelos, *notebooks* ou scripts de avaliação, lista de consultas e *relevance judgments* produzidos. Inclua um README com instruções de reprodução.
3. **Demonstração mínima de uso.** Um *script* ou *notebook* que, dada uma consulta em texto, retorna a lista ranqueada de documentos - para que o professor consiga executar pelo menos uma consulta-exemplo.
4. **Vídeo de apresentação.** Vídeo de até **8 minutos** apresentando o trabalho: motivação, decisões de projeto (coleta, pré-processamento, modelos), metodologia experimental, resultados e principais lições aprendidas. Postar em plataforma de acesso público (YouTube ou Google Drive); incluir o link em nota de rodapé na conclusão do relatório.

8. Critérios de correção

A nota final será composta de acordo com a rubrica abaixo (total 100 pontos). Para cada critério, considere os níveis de desempenho: *Insuficiente* (0–40% dos pontos), *Regular* (40–70%), *Bom* (70–90%) e *Excelente* (90–100%).

¹<http://www.sbc.org.br/documentos-da-sbc/category/169-templates-para-artigos-e-capitulos-de-livros>

#	Critério	Pontos
C1	Coleta e documentação da coleção: tamanho adequado, representatividade do tema, qualidade dos metadados, reprodutibilidade da coleta.	10
C2	Pré-processamento: adequação das escolhas, discussão de impactos, qualidade do código.	5
C3	BM25 (baseline): corretude da implementação/uso, justificativa de hiperparâmetros, conexão com paradigma probabilístico.	10
C4	Recuperação por KNN/densa: corretude, escolha da representação, comparação com o baseline.	10
C5	Módulo(s) de aprofundamento: qualidade técnica, integração com o pipeline, profundidade de análise. (M: até 15 pts; D: até 20 pts distribuídos nos 2 módulos.)	15–20
C6	Avaliação experimental: construção e qualidade do conjunto de <i>queries+qrels</i> , uso correto de métricas (P@k, R@k, MAP/nDCG), análise qualitativa. <i>Para Doutorado: teste estatístico de significância.</i>	15
C7	Discussão crítica e conexão com a disciplina: relação explícita entre componentes do sistema e tópicos vistos em sala; reflexão sobre limites e <i>trade-offs</i> .	10
C8	Qualidade do relatório: clareza da escrita, formato SBC respeitado, figuras e tabelas legíveis, referências adequadas e atuais.	10
C9	Reprodutibilidade do código: README, dependências declaradas, instruções claras, execução sem ajustes manuais excessivos.	5
C10	Vídeo de apresentação: dentro de 8 minutos, clareza da exposição, cobertura dos pontos exigidos (motivação, método, resultados).	10
Total		100

Penalidades. Atraso na entrega: zera a nota do trabalho (entregas após as 23:59 do prazo *não* serão aceitas; em caso de dificuldades, procure o professor com antecedência). Vídeo acima de 8 min: –5 pontos. Ausência de código/dados ou impossibilidade de reproduzir resultados: zera o critério C9 e reduz pela metade o critério C6.

9. Submissão

O prazo de entrega é até as **23:59 do dia 19/06/2026**, no AVA da disciplina. Submeter um único arquivo `.zip` contendo o relatório em PDF, o código-fonte, os *scripts*/dados de avaliação (*queries* e *qrels*) e um arquivo `LINKS.txt` com URLs do vídeo e, se aplicável, do repositório. Caso a coleção bruta seja grande, é permitido disponibilizá-la externamente (link no `LINKS.txt`).

10. Materiais de apoio fornecidos

Junto a este enunciado, são disponibilizados os seguintes materiais (todos opcionais; o uso, contudo, é fortemente recomendado para que o seu esforço seja direcionado às decisões interessantes do trabalho):

- `coleta_arxiv.ipynb` - notebook base de coleta de dados via API do ArXiv (configurável para o seu tema de pesquisa, com salvamento incremental e *sanity checks*).
- `material_de_apoio/baseline_bm25.ipynb` - baseline BM25 mínimo viável sobre o `corpus.jsonl`, com função de busca e geração de arquivo de *run* no formato TREC.
- `material_de_apoio/eval/` - template de `queries.tsv` e `qrels.tsv`, e script `evaluate.py` que calcula P@k, R@k, MAP e nDCG sobre arquivos de *run* no formato TREC.

- `material_de_apoio/skeleton/` - estrutura sugerida de repositório com `README.md`, `requirements.txt`, `CHECKLIST.md` de submissão e `LINKS.txt`.
- `material_de_apoio/Leituras_deReferencia.md` - lista curta de leituras recomendadas, com indicação de como cada uma se conecta ao trabalho e à disciplina.

11. Leituras recomendadas

A lista abaixo é **curada e não exaustiva**: foram selecionadas leituras que dão âncora bibliográfica ao trabalho e que mapeiam diretamente para os tópicos da disciplina. Não é necessário ler todas integralmente - foque nas partes relacionadas aos componentes que você implementar e cite-as no relatório quando pertinente, junto a outras referências que você encontrar.

Recuperação esparsa (BM25, TF-IDF).

- Robertson, S. & Zaragoza, H. (2009). *The Probabilistic Relevance Framework: BM25 and Beyond*. Foundations and Trends in Information Retrieval, 3(4), 333–389. - Texto canônico sobre BM25, com a derivação do modelo probabilístico que conecta diretamente com o paradigma de Naïve Bayes visto em aula.
- Manning, C. D., Raghavan, P. & Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge University Press. Caps. 6 (TF-IDF) e 11 (modelos probabilísticos). Disponível em <https://nlp.stanford.edu/IR-book/>.

Recuperação densa e híbrida.

- Karpukhin, V. et al. (2020). *Dense Passage Retrieval for Open-Domain Question Answering*. EMNLP.
- Cohan, A. et al. (2020). *SPECTER: Document-level Representation Learning using Citation-informed Transformers*. ACL. (Veja também *SPECTER2*, 2023.)
- Lin, J. (2021). *A Proposed Conceptual Framework for a Representational Approach to Information Retrieval*. SIGIR Forum, 55(2). - Visão unificada de *sparse*, *dense* e híbrido; excelente para a seção de Trabalhos Relacionados.

Avaliação e *benchmarks*.

- Thakur, N. et al. (2021). *BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models*. NeurIPS Datasets & Benchmarks. - Útil para inspirar a metodologia experimental (lembre-se: BEIR não pode ser a sua coleção principal - ver Seção 5).

Contextualização do componente “R” do RAG.

- Lewis, P. et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. NeurIPS. - Paper que cunhou o termo *RAG*.
- Gao, Y. et al. (2024). *Retrieval-Augmented Generation for Large Language Models: A Survey*. arXiv:2312.10997.

Conexões com a disciplina.

- Naïve Bayes ↔ BM25: Manning, Raghavan & Schütze, Cap. 11.
- KNN ↔ recuperação densa: Manning, Raghavan & Schütze, Cap. 14 + slides da aula de KNN.
- Algoritmos de Otimização ↔ ajuste de hiperparâmetros: slides da disciplina + Bergstra, J. & Bengio, Y. (2012), *Random Search for Hyper-Parameter Optimization*, JMLR.
- Agrupamento de Dados ↔ organização de resultados: slides da disciplina + Aggarwal, C. C. & Reddy, C. K. (2013), *Data Clustering: Algorithms and Applications*, Cap. 17.

- Regras de Associação $\leftrightarrow$ expansão de consulta: slides da disciplina + Han, J., Pei, J. & Kamber, M. (2011), *Data Mining: Concepts and Techniques*.

12. Observações finais

É **esperado** o uso de bibliotecas estabelecidas (`rank_bm25`, `scikit-learn`, `sentence-transformers`, `numpy`, `pandas`, `nltk/spaCy`, etc.). Não é necessário implementar tudo do zero - o foco é a *combinação inteligente* das técnicas e a *análise crítica* dos resultados. Não é permitido, contudo, simplesmente acoplar uma API de RAG comercial “caixa-preta”: as decisões de modelagem devem ser suas.

O uso de assistentes de IA generativa é permitido para apoio à escrita e à programação, desde que *declarado* no relatório (uma seção curta indicando onde e como foram utilizados). Plágio acarretará nota zero.

Quaisquer dúvidas, não hesitem em procurar o professor.

Bom trabalho!